SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956

ENATHUR, KANCHIPURAM - 631 561 DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



NAME : THULASIPRASATH T

REG NO : 11249A383

SECTION : S4

SUBJECT CODE : BCSF245T40

SUBJECT NAME : DATABASE MANAGEMENT SYSTEM (DBMS)

SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956

ENATHUR, KANCHIPURAM - 631 561 DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by

Mr/Ms.THULASIPRASATH T with Reg. No 11249A383 done by of II-B.E.(CSE) in – DATABASE MANAGEMENT SYSTEMS LAB

for during the academic year 2025 -2026.

Station : ENATHUR

Date :

Staff-in-charge **Head of the Department**

Submitted for the practical examination held on _____

Examine-1 **Examine -2**

INDEX

SN. NO.	PROGRAM NAME	PG. NUMBER	DATE	SIGNATURE
1	Student, Employee and Library Databases	1		
2	Constraints	4		
3	DCL and TCL Commands	6		
4	Aggregate Functions, GROUP BY and HAVING	8		
5	Relationships, Joins and Subqueries	11		
6	INSERT, DELETE, ALTER, UPDATE and SELECT	14		
7	ANY, ALL, IN, EXISTS, NOT EXISTS, UNION and INTERSECT	16		
8	Views, Sequences, Indexes and Save points	20		
9	Normalization (1NF, 2NF, 3NF, BCNF, 4NF and 5NF)	22		
10	PL/SQL: Conditions, Exceptions, Procedures, Functions, Triggers and Cursors	24		
11	Database Design Case Studies	29		

EXP-1 **Creation of Student, Employee and Library Databases.**

AIM

To create Student, Employee and Library tables, insert records, and retrieve the stored information.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment.
- SQL*Plus or Oracle SQL Developer.
- DBMS_OUTPUT enabled for PL/SQL exercises.

ALGORITHM

Step1: Start the SQL environment and select the working schema.

Step2:Create the required table with suitable columns and data types.

Step3:Insert sample records into each table.

Step4:Execute SELECT queries to display the stored records.

Step:Verify the displayed rows and finish.

PROGRAM / SQL COMMANDS

1(a) Student database

```
CREATE TABLE STUDENT (
SID NUMBER(3),
NAME VARCHAR2(20),
MARKS NUMBER(3)
);
INSERT INTO STUDENT VALUES (101, 'RAVI', 85);
INSERT INTO STUDENT VALUES (102, 'RAJU', 75);
INSERT INTO STUDENT VALUES (103, 'SITA', 90);
SELECT * FROM STUDENT;
```

1(b) Employee database

```
CREATE TABLE EMPLOYEE (
EID NUMBER(3),
NAME VARCHAR2(20),
SALARY NUMBER(6)
);
INSERT INTO EMPLOYEE VALUES (101, 'RAVI', 25000);
INSERT INTO EMPLOYEE VALUES (102, 'RAJU', 30000);
INSERT INTO EMPLOYEE VALUES (103, 'SITA', 28000);
SELECT * FROM EMPLOYEE;
```

1(c) Library database

```
CREATE TABLE LIBRARY (
BOOKID NUMBER(3),
TITLE VARCHAR2(20),
PRICE NUMBER(5)
);
INSERT INTO LIBRARY VALUES (101, 'DBMS', 300);
INSERT INTO LIBRARY VALUES (102, 'JAVA', 250);
INSERT INTO LIBRARY VALUES (103, 'CN', 280);
SELECT * FROM LIBRARY;
```

OUTPUT

STUDENT

```
SID | NAME | MARKS
----+------+------
101 | RAVI | 85
102 | RAJU | 75
103 | SITA | 90
```

EMPLOYEE

EID | NAME | SALARY

----+------+-------

101 | RAVI | 25000

102 | RAJU | 30000

103 | SITA | 28000

LIBRARY

BOOKID | TITLE | PRICE

-------+-------+------

101 | DBMS | 300

102 | JAVA | 250

103 | CN | 280

RESULT

Thus, the Student, Employee and Library tables were created, populated, and queried successfully.

Exp-2 **Creating a Table with Constraints.**

AIM

To create a table using PRIMARY KEY, NOT NULL, CHECK and UNIQUE constraints.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment
- SQL*Plus or Oracle SQL Developer
- DBMS_OUTPUT enabled for PL/SQL exercises

ALGORITHM

Step1: Start the Oracle SQL environment.

Step2: Create the STUDENT1 table with the required constraints.

Step3: Insert records that satisfy each constraint.

Step4: Display the inserted records using SELECT.

Step5: Confirm that the table enforces the defined constraints.

PROGRAM / SQL COMMANDS

```
CREATE TABLE STUDENT1 (
ROLLNO NUMBER(3) PRIMARY KEY,
NAME VARCHAR2(20) NOT NULL,
MARKS NUMBER(3) CHECK (MARKS >= 0),
EMAIL VARCHAR2(30) UNIQUE
);

INSERT INTO STUDENT1 VALUES (101, 'RAVI', 85, 'ravi@gmail.com');
INSERT INTO STUDENT1 VALUES (102, 'RAJU', 75, 'raju@gmail.com');
INSERT INTO STUDENT1 VALUES (103, 'SITA', 90, 'sita@gmail.com');

SELECT * FROM STUDENT1;
```

OUTPUT

SELECT * FROM STUDENT1

ROLLNO | NAME | MARKS | EMAIL

-------+------+-------+---------------

101 | RAVI | 85 | ravi@gmail.com

102 | RAJU | 75 | raju@gmail.com

103 | SITA | 90 | sita@gmail.com

RESULT

Thus, the STUDENT1 table was created and records were inserted using the specified constraints.

Exp-3 **DCL and TCL Commands**

AIM

To demonstrate Data Control Language (DCL) and Transaction Control Language (TCL) commands.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment
- SQL*Plus or Oracle SQL Developer
- DBMS_OUTPUT enabled for PL/SQL exercises

ALGORITHM

Step1:Create or select the example STUDENT table and ensure the required user/privileges exist.

Step2:Use GRANT to provide a privilege to a database user.

Step3:Use REVOKE to remove that privilege.

Step4:Make a data change and set a savepoint within a transaction.

Step5: Demonstrate rollback and commit, then verify the final state.

PROGRAM / SQL COMMANDS

DCL:

run using an account that owns STUDENT and can grant privileges.

```
GRANT SELECT ON STUDENT TO SYSTEM;
REVOKE SELECT ON STUDENT FROM SYSTEM;
```

TCL:

demonstrate savepoint and rollback on a sample row.

```
UPDATE STUDENT SET MARKS = 95 WHERE ROLLNO = 101;
SAVEPOINT A;
UPDATE STUDENT SET MARKS = 80 WHERE ROLLNO = 101;
ROLLBACK TO A;
```

```
SELECT ROLLNO, MARKS FROM STUDENT WHERE ROLLNO = 101;
COMMIT;
```

OUTPUT

```
After ROLLBACK TO A
ROLLNO | MARKS
-----------+------
101        | 95
```

RESULT

Thus, DCL and TCL commands were demonstrated successfully.

Exp-4 **Aggregate Functions, GROUP BY and HAVING**

AIM

To execute aggregate functions and SQL queries using GROUP BY and HAVING.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment.
- SQL*Plus or Oracle SQL Developer.
- DBMS_OUTPUT enabled for PL/SQL exercises.

ALGORITHM

Step1: Start the SQL environment and create/populate a STUDENT table with ROLLNO and MARKS columns.

Step2: Execute COUNT, SUM, AVG, MAX and MIN aggregate functions.

Step3: Add a department column and populate it for each row.

Step4: Group the rows by department and count records in each group.

Step5: Use HAVING to filter groups and inspect the results.

PROGRAM / SQL COMMANDS

```
-- Example assumes STUDENT(ROLLNO, NAME, MARKS) has three sample rows:
-- (101, 'RAVI', 85), (102, 'RAJU', 75), (103, 'SITA', 90)
SELECT COUNT(*) AS TOTAL_STUDENTS FROM STUDENT;
SELECT SUM(MARKS) AS TOTAL_MARKS FROM STUDENT;
SELECT AVG(MARKS) AS AVERAGE_MARKS FROM STUDENT;
SELECT MAX(MARKS) AS HIGHEST_MARKS FROM STUDENT;
SELECT MIN(MARKS) AS LOWEST_MARKS FROM STUDENT;

ALTER TABLE STUDENT ADD DEPT VARCHAR2(10);
UPDATE STUDENT SET DEPT = 'CSE' WHERE ROLLNO = 101;
UPDATE STUDENT SET DEPT = 'CSE' WHERE ROLLNO = 102;
```

```
UPDATE STUDENT SET DEPT = 'ECE' WHERE ROLLNO = 103;

SELECT DEPT, COUNT(*) AS STUDENT_COUNT
FROM STUDENT
GROUP BY DEPT;

SELECT DEPT, COUNT(*) AS STUDENT_COUNT
FROM STUDENT
GROUP BY DEPT
HAVING COUNT(*) > 1;
```

OUTPUT

Aggregate function outputs

```
TOTAL_STUDENTS
--------------
3
TOTAL_MARKS
-----------
250
AVERAGE_MARKS
-------------
83.333
HIGHEST_MARKS
-------------
90
LOWEST_MARKS
------------
75
```

GROUP BY

```
DEPT | STUDENT_COUNT
-----+--------------
```

CSE | 2
ECE | 1

HAVING COUNT(*) > 1

DEPT | STUDENT_COUNT
-----+--------------
CSE | 2

RESULT

Thus, aggregate functions, GROUP BY and HAVING were executed successfully.

Exp-5 **Relationships, Joins and Subqueries**

AIM

To define a relationship between STUDENT and COURSE tables and execute JOIN and subquery operations.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment.
- SQL*Plus or Oracle SQL Developer.
- DBMS_OUTPUT enabled for PL/SQL exercises.

ALGORITHM

Step1:Create the COURSE table with CID as its primary key.

Step2:Create the STUDENT table and use CID to reference COURSE.

Step3:Insert sample course and student records.

Step4:Execute INNER JOIN and LEFT JOIN queries to combine related data.

Step5:Execute a subquery to retrieve students enrolled in the CSE course.

PROGRAM / SQL COMMANDS

```
CREATE TABLE COURSE (
CID NUMBER(3) PRIMARY KEY,
CNAME VARCHAR2(20)
);

CREATE TABLE STUDENT (
SID NUMBER(3) PRIMARY KEY,
NAME VARCHAR2(20),
CID NUMBER(3),
CONSTRAINT FK_STUDENT_COURSE FOREIGN KEY (CID) REFERENCES
COURSE(CID)
);
```

```
INSERT INTO COURSE VALUES (1, 'CSE');
INSERT INTO COURSE VALUES (2, 'ECE');
INSERT INTO COURSE VALUES (3, 'AI');

INSERT INTO STUDENT VALUES (101, 'RAVI', 1);
INSERT INTO STUDENT VALUES (102, 'RAJU', 2);
INSERT INTO STUDENT VALUES (103, 'SITA', 1);

-- INNER JOIN
SELECT S.NAME, C.CNAME
FROM STUDENT S
INNER JOIN COURSE C ON S.CID = C.CID;

-- LEFT JOIN
SELECT S.NAME, C.CNAME
FROM STUDENT S
LEFT JOIN COURSE C ON S.CID = C.CID;

-- SUBQUERY: students in CSE
SELECT NAME
FROM STUDENT
WHERE CID = (SELECT CID FROM COURSE WHERE CNAME = 'CSE');
```

OUTPUT

INNER JOIN

```
NAME | CNAME
-----+------
RAVI | CSE
RAJU | ECE
SITA | CSE
```

LEFT JOIN

NAME | CNAME

-----+------

RAVI | CSE

RAJU | ECE

SITA | CSE

Subquery: students in CSE

NAME

RAVI

SITA

RESULT

Thus, the relationship, JOIN operations and subquery were demonstrated successfully.

Exp-6 INSERT, DELETE, ALTER, UPDATE and SELECT

AIM

To perform INSERT, UPDATE, DELETE, ALTER and SELECT operations on a table.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment.
- SQL*Plus or Oracle SQL Developer.
- DBMS_OUTPUT enabled for PL/SQL exercises.

ALGORITHM

Step1: Create a STUDENT table with ROLLNO, NAME and MARKS.

Step2: Insert three sample student records.

Step3: Display all records using SELECT.

Step4: Update a student mark and delete one student row.

Step5: Alter the table to add AGE, then display the remaining records.

PROGRAM / SQL COMMANDS

```
CREATE TABLE STUDENT (
ROLLNO NUMBER(3),
NAME VARCHAR2(20),
MARKS NUMBER(3)
);

INSERT INTO STUDENT VALUES (101, 'RAVI', 85);
INSERT INTO STUDENT VALUES (102, 'RAJU', 75);
INSERT INTO STUDENT VALUES (103, 'SITA', 90);

SELECT * FROM STUDENT;

UPDATE STUDENT SET MARKS = 80 WHERE ROLLNO = 102;
```

```
DELETE FROM STUDENT WHERE ROLLNO = 103;
ALTER TABLE STUDENT ADD AGE NUMBER(2);

SELECT * FROM STUDENT;
```

OUTPUT

Initial SELECT

```
ROLLNO | NAME | MARKS
-------+------+------
101 | RAVI | 85
102 | RAJU | 75
103 | SITA | 90
```

Final SELECT after UPDATE, DELETE and ALTER

```
ROLLNO | NAME | MARKS | AGE
-------+------+-------+-----
101 | RAVI | 85 | NULL
102 | RAJU | 80 | NULL
```

RESULT

Thus, INSERT, UPDATE, DELETE, ALTER and SELECT operations were performed successfully.

Exp7 ANY, ALL, IN, EXISTS, NOT EXISTS, UNION and INTERSECT

AIM

To execute SQL queries using IN, ANY, ALL, EXISTS, NOT EXISTS, UNION and INTERSECT.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment
- SQL*Plus or Oracle SQL Developer
- DBMS_OUTPUT enabled for PL/SQL exercises

ALGORITHM

Step1: Create and populate STUDENT and STUDENT2 tables.

Step2: Use IN to select rows matching a list of marks.

Step3: Use ANY and ALL to compare a value with the results of a subquery.

Step4: Use EXISTS and NOT EXISTS to test matching keys in the other table.

Step5: Use UNION and INTERSECT to combine or compare names from both tables.

PROGRAM / SQL COMMANDS

```
CREATE TABLE STUDENT (
ROLLNO NUMBER(3), NAME VARCHAR2(20), MARKS NUMBER(3)
);
INSERT INTO STUDENT VALUES (101, 'RAVI', 85);
INSERT INTO STUDENT VALUES (102, 'RAJU', 75);
INSERT INTO STUDENT VALUES (103, 'SITA', 90);

CREATE TABLE STUDENT2 (
ROLLNO NUMBER(3), NAME VARCHAR2(20), MARKS NUMBER(3)
);
INSERT INTO STUDENT2 VALUES (101, 'RAVI', 85);
INSERT INTO STUDENT2 VALUES (104, 'RAM', 80);
```

```
INSERT INTO STUDENT2 VALUES (105, 'KIRAN', 70);

-- IN
SELECT * FROM STUDENT WHERE MARKS IN (75, 90);

-- ANY
SELECT * FROM STUDENT
WHERE MARKS > ANY (SELECT MARKS FROM STUDENT2);

-- ALL
SELECT * FROM STUDENT
WHERE MARKS > ALL (SELECT MARKS FROM STUDENT2);

-- EXISTS
SELECT * FROM STUDENT S
WHERE EXISTS (SELECT 1 FROM STUDENT2 S2 WHERE S.ROLLNO =
S2.ROLLNO);

-- NOT EXISTS
SELECT * FROM STUDENT S
WHERE NOT EXISTS (SELECT 1 FROM STUDENT2 S2 WHERE S.ROLLNO =
S2.ROLLNO);

-- UNION
SELECT NAME FROM STUDENT
UNION
SELECT NAME FROM STUDENT2;

-- INTERSECT
SELECT NAME FROM STUDENT
INTERSECT
SELECT NAME FROM STUDENT2;
```

OUTPUT

IN (75, 90)

ROLLNO | NAME | MARKS

-------+------+------

102 | RAJU | 75

103 | SITA | 90

ANY equivalent (marks greater than at least one mark in STUDENT2)

ROLLNO | NAME | MARKS

-------+------+------

101 | RAVI | 85

102 | RAJU | 75

103 | SITA | 90

ALL equivalent (marks greater than every mark in STUDENT2)

ROLLNO | NAME | MARKS

-------+------+------

103 | SITA | 90

EXISTS

ROLLNO | NAME | MARKS

-------+------+------

101 | RAVI | 85

NOT EXISTS

ROLLNO | NAME | MARKS

-------+------+------

102 | RAJU | 75

103 | SITA | 90

UNION

NAME

KIRAN

RAJU
RAM
RAVI
SITA

INTERSECT

NAME

RAVI

RESULT

Thus, the specified comparison, existence and set operators were demonstrated successfully.

Exp-8 **Views, Sequences, Indexes and Save points**

AIM

To create and use a view, sequence and index, and demonstrate a transaction savepoint.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment
- SQL*Plus or Oracle SQL Developer
- DBMS_OUTPUT enabled for PL/SQL exercises

ALGORITHM

Step1: Create a STUDENT table and insert sample rows.

Step2: Create a view and query the view.

Step3: Create a sequence and retrieve its next value.

Step4: Create an index on the NAME column.

Step5: Update records around a savepoint, roll back to the savepoint and display final data.

PROGRAM / SQL COMMANDS

```
CREATE TABLE STUDENT (
ROLLNO NUMBER(3), NAME VARCHAR2(20), MARKS NUMBER(3)
);
INSERT INTO STUDENT VALUES (101, 'RAVI', 85);
INSERT INTO STUDENT VALUES (102, 'RAJU', 75);
INSERT INTO STUDENT VALUES (103, 'SITA', 90);

-- VIEW
CREATE VIEW STUDENT_VIEW AS
SELECT ROLLNO, NAME, MARKS FROM STUDENT;
SELECT * FROM STUDENT_VIEW;
```

```
-- SEQUENCE
CREATE SEQUENCE STUDENT_SEQ START WITH 104 INCREMENT BY 1;
SELECT STUDENT_SEQ.NEXTVAL FROM DUAL;

-- INDEX
CREATE INDEX STUDENT_INDEX ON STUDENT(NAME);

-- SAVEPOINT
UPDATE STUDENT SET MARKS = 95 WHERE ROLLNO = 101;
SAVEPOINT S1;
UPDATE STUDENT SET MARKS = 70 WHERE ROLLNO = 102;
ROLLBACK TO S1;
SELECT * FROM STUDENT;
COMMIT;
```

OUTPUT

```
ROLLNO | NAME | MARKS
-------+------+------
101 | RAVI | 95
102 | RAJU | 75
103 | SITA | 90

NEXTVAL
104
```

RESULT

Thus, the view, sequence and index were created and the savepoint/rollback behavior was demonstrated successfully.

Exp-9 Database Normalization from 1NF to 5NF

AIM

To study database normalization and decompose relations to reduce redundancy and update anomalies, up to 1NF, 2NF, 3NF, 4NF and 5NF.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment
- SQL*Plus or Oracle SQL Developer
- DBMS_OUTPUT enabled for PL/SQL exercises

ALGORITHM

Step1: Identify the relation attributes, candidate keys and dependencies from the given scenario.

Step2: Convert the unnormalized relation to 1NF by storing atomic values only.

Step3: Convert to 2NF by removing partial dependencies on part of a composite candidate key.

Step4: Convert to 3NF by removing transitive dependencies of non-key attributes.

Step5: Check for non-trivial multivalued dependencies (4NF) and join dependencies (5NF), decomposing only where justified.

PROGRAM / SQL COMMANDS

Illustrative relation:
STUDENT_COURSE_PHONE(SID, SNAME, COURSE, PHONE)

Example functional dependency:
SID -> SNAME

If each student may take many courses and have many phone numbers independently,

the combined rows can repeat student details and create update anomalies.

Possible decomposition:
STUDENT(SID, SNAME)
STUDENT_COURSE(SID, COURSE)
STUDENT_PHONE(SID, PHONE)

1NF: every attribute value is atomic.
2NF: every non-prime attribute is fully functionally dependent on each candidate key.
3NF: for every non-trivial FD X -> A, X is a superkey or A is prime (standard 3NF test).
4NF: every non-trivial multivalued dependency has a superkey determinant.
5NF: every non-trivial join dependency is implied by candidate keys.

OUTPUT

1NF: each attribute contains an atomic value; no repeating groups.
2NF: remove partial dependencies of non-prime attributes on part of a candidate key.
3NF: for each non-trivial functional dependency X -> A, X is a superkey or A is prime.
4NF: every non-trivial multivalued dependency must have a superkey determinant.
5NF: every non-trivial join dependency must be implied by candidate keys.
Example decomposition:
STUDENT(SID, SNAME)
STUDENT_COURSE(SID, COURSE)
STUDENT_PHONE(SID, PHONE)

RESULT

Thus, the principles and dependency tests used for normalization through 5NF were studied.

Exp-10 PL/SQL Conditions, Exceptions, Procedures, Functions, Triggers and Cursors

AIM

To execute PL/SQL programs using conditional statements, exception handling, procedures, functions, database triggers, and implicit and explicit cursors.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment
- SQL*Plus or Oracle SQL Developer
- DBMS_OUTPUT enabled for PL/SQL exercises

ALGORITHM

Step1: Enable server output in SQL*Plus or SQL Developer.

Step2: Write and execute a PL/SQL block with an IF conditional statement.

Step3: Handle the ZERO_DIVIDE exception in a PL/SQL block.

Step4: Create and call a stored procedure and create/query a stored function.

Step5: Create a table and triggers, then demonstrate implicit and explicit cursor processing.

PROGRAM / SQL COMMANDS

```
SET SERVEROUTPUT ON;


-- 10(a) Conditional statement
DECLARE
N NUMBER := &N;
BEGIN
IF N > 0 THEN
DBMS_OUTPUT.PUT_LINE('Positive Number');
ELSIF N = 0 THEN
```

```
DBMS_OUTPUT.PUT_LINE('Zero');
ELSE
DBMS_OUTPUT.PUT_LINE('Negative Number');
END IF;
END;
/

-- 10(b) Exception handling
DECLARE
A NUMBER := 10;
B NUMBER := 0;
C NUMBER;
BEGIN
C := A / B;
DBMS_OUTPUT.PUT_LINE(C);
EXCEPTION
WHEN ZERO_DIVIDE THEN
DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');
END;
/

-- 10(c) Procedure
CREATE OR REPLACE PROCEDURE GREET AS
BEGIN
DBMS_OUTPUT.PUT_LINE('Hello Student');
END;
/
EXEC GREET;

-- Function
CREATE OR REPLACE FUNCTION ADD_NUM
RETURN NUMBER AS
```

```
BEGIN
RETURN 10 + 20;
END;
/
SELECT ADD_NUM() AS RESULT FROM DUAL;

-- 10(d) Triggers: create table before compiling triggers
CREATE TABLE STUDENT (
ROLLNO NUMBER(3), NAME VARCHAR2(20), MARKS NUMBER(3)
);
CREATE OR REPLACE TRIGGER STUDENT_INSERT
AFTER INSERT ON STUDENT
BEGIN
DBMS_OUTPUT.PUT_LINE('Record Inserted');
END;
/
CREATE OR REPLACE TRIGGER STUDENT_UPDATE
AFTER UPDATE ON STUDENT
BEGIN
DBMS_OUTPUT.PUT_LINE('Record Updated');
END;
/
CREATE OR REPLACE TRIGGER STUDENT_DELETE
AFTER DELETE ON STUDENT
BEGIN
DBMS_OUTPUT.PUT_LINE('Record Deleted');
END;
/
INSERT INTO STUDENT VALUES (101, 'RAVI', 85);
UPDATE STUDENT SET MARKS = 90 WHERE ROLLNO = 101;
DELETE FROM STUDENT WHERE ROLLNO = 101;
```

```
-- 10(e) Implicit cursor
BEGIN
UPDATE STUDENT SET MARKS = 90 WHERE ROLLNO = 101;
DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' row(s) updated');
END;
/

-- Explicit cursor (insert rows before running this block if needed)
INSERT INTO STUDENT VALUES (101, 'RAVI', 85);
INSERT INTO STUDENT VALUES (102, 'RAJU', 75);

DECLARE
CURSOR C1 IS SELECT NAME, MARKS FROM STUDENT ORDER BY ROLLNO;
V_NAME STUDENT.NAME%TYPE;
V_MARKS STUDENT.MARKS%TYPE;
BEGIN
OPEN C1;
LOOP
FETCH C1 INTO V_NAME, V_MARKS;
EXIT WHEN C1%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(V_NAME || ' ' || V_MARKS);
END LOOP;
CLOSE C1;
END;
```

OUTPUT

10(a) Conditional statement (sample input N = 7)

Positive Number

10(b) Exception handling

Cannot divide by zero

10(c) Procedure GREET

Hello Student

Function ADD_NUM

RESULT

30

10(d) Trigger messages (expected in Oracle with DBMS_OUTPUT enabled)

Record Inserted
Record Updated
Record Deleted

10(e) Implicit cursor

1 row(s) updated — only if a matching ROLLNO exists at the time of the UPDATE.

Explicit cursor

RAVI 85
RAJU 75

RESULT

Thus, PL/SQL conditions, exception handling, procedures, functions, triggers, and implicit and explicit cursor concepts were demonstrated.

Exp-11 **Database Design Case Studies**

AIM

To create example database tables for Student Information, Hospital Management, Railway Reservation, Timetable Management and Hotel Management systems.

SOFTWARE REQUIRED

- Oracle Database or an Oracle-compatible SQL lab environment
- SQL*Plus or Oracle SQL Developer
- DBMS_OUTPUT enabled for PL/SQL exercises

ALGORITHM

Step1: Identify the main entity and attributes for each case study.

Step2: Choose an appropriate primary key for each table.

Step3: Create the tables using suitable Oracle data types.

Step4: Insert sample records into each table.

Step5: Execute SELECT queries to display the stored data.

PROGRAM / SQL COMMANDS

```
-- 11(a) Student Information System
CREATE TABLE STUDENT_INFO (
SID NUMBER(3) PRIMARY KEY,
NAME VARCHAR2(20),
MARKS NUMBER(3)
);
INSERT INTO STUDENT_INFO VALUES (101, 'RAVI', 85);
INSERT INTO STUDENT_INFO VALUES (102, 'RAJU', 75);
INSERT INTO STUDENT_INFO VALUES (103, 'SITA', 90);
SELECT * FROM STUDENT_INFO;

-- 11(b) Hospital Management System
```

```
CREATE TABLE PATIENT (
PID NUMBER(3) PRIMARY KEY,
NAME VARCHAR2(20),
DISEASE VARCHAR2(20)
);
INSERT INTO PATIENT VALUES (101, 'RAVI', 'FEVER');
INSERT INTO PATIENT VALUES (102, 'RAJU', 'COLD');
INSERT INTO PATIENT VALUES (103, 'SITA', 'MALARIA');
SELECT * FROM PATIENT;

-- 11(c) Railway Reservation System
CREATE TABLE RESERVATION (
TICKET_NO NUMBER(3) PRIMARY KEY,
NAME VARCHAR2(20),
TRAIN VARCHAR2(20)
);
INSERT INTO RESERVATION VALUES (101, 'RAVI', 'CHENNAI');
INSERT INTO RESERVATION VALUES (102, 'RAJU', 'BANGALORE');
INSERT INTO RESERVATION VALUES (103, 'SITA', 'HYDERABAD');
SELECT * FROM RESERVATION;

-- 11(d) Timetable Management System
CREATE TABLE TIMETABLE (
DAY VARCHAR2(10),
SUBJECT VARCHAR2(20),
ROOM NUMBER(3)
);
INSERT INTO TIMETABLE VALUES ('MONDAY', 'JAVA', 101);
INSERT INTO TIMETABLE VALUES ('TUESDAY', 'DBMS', 102);
INSERT INTO TIMETABLE VALUES ('WEDNESDAY', 'CN', 103);
SELECT * FROM TIMETABLE;
```

```
-- 11(e) Hotel Management System
CREATE TABLE HOTEL (
ROOM_NO NUMBER(3) PRIMARY KEY,
GUEST_NAME VARCHAR2(20),
ROOM_TYPE VARCHAR2(20)
);
INSERT INTO HOTEL VALUES (101, 'RAVI', 'SINGLE');
INSERT INTO HOTEL VALUES (102, 'RAJU', 'DOUBLE');
INSERT INTO HOTEL VALUES (103, 'SITA', 'DELUXE');
SELECT * FROM HOTEL;
```

OUTPUT

11(a) STUDENT_INFO

```
SID | NAME | MARKS
----+------+------
101 | RAVI | 85
102 | RAJU | 75
103 | SITA | 90
```

11(b) PATIENT

```
PID | NAME | DISEASE
----+------+--------
101 | RAVI | FEVER
102 | RAJU | COLD
103 | SITA | MALARIA
```

11(c) RESERVATION

```
TICKET_NO | NAME | TRAIN
----------+------+----------
101 | RAVI | CHENNAI
102 | RAJU | BANGALORE
103 | SITA | HYDERABAD
```

11(d) TIMETABLE

DAY | SUBJECT | ROOM

----------+---------+-----

MONDAY | JAVA | 101

TUESDAY | DBMS | 102

WEDNESDAY | CN | 103

11(e) HOTEL

ROOM_NO | GUEST_NAME | ROOM_TYPE

--------+------------+----------

101 | RAVI | SINGLE

102 | RAJU | DOUBLE

103 | SITA | DELUXE

RESULT

Thus, the five example database designs were created and sample records were inserted and retrieved successfully.